ASSIGNMENT 4 – CO4

Designing Reusable Class Hierarchies

DSA01 · C++ Object-Oriented Programming · 50 Marks

ASSIGNMENT 4 – CO4

Title: Designing Reusable Class Hierarchies

- Course: Data Structures and Algorithms / C++ Programming
- Course Code: DSA01
- Assessment: Assignment 4
- Maximum Marks: 50
- Student Name: SK ALI BABU
- Register Number: 192524038
- Git hub link : https://github.com/shaikalibabu143-alt/DSA0104-C-

Task 1: Designing a Payroll Class Hierarchy

1. Aim

To design a reusable class hierarchy for calculating employee salaries using inheritance, data hiding, and runtime polymorphism.

2. Class hierarchy diagram

Employee

employeeId, name, virtual calculateSalary()

↓ Inheritance

FullTimeEmployee

Basic salary, allowance

PartTimeEmployee

Hours worked, hourly rate

3. Explanation

The Employee class acts as the base class. It stores common employee details and declares calculateSalary() as a virtual function.

- FullTimeEmployee calculates salary using basic pay and allowance.
- PartTimeEmployee calculates salary using hours worked and hourly rate.
- Runtime polymorphism allows the correct salary function to execute through a base-class pointer.
- Private data members protect employee information.

4. C++ program

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class Employee {
protected:
   int employeeId;
```

```
    string name;

public:
    Employee(int id, string n) {
        employeeId = id;
        name = n;
    }

    virtual double calculateSalary() const = 0;

    virtual void display() const {
        cout << "Employee ID: " << employeeId << endl;
        cout << "Name: " << name << endl;
    }

    virtual ~Employee() {}
};

class FullTimeEmployee : public Employee {
private:
    double basicSalary;
    double allowance;

public:
    FullTimeEmployee(int id, string n,
                     double basic, double allow)
        : Employee(id, n) {
        basicSalary = basic;
        allowance = allow;
    }

    double calculateSalary() const override {
        return basicSalary + allowance;
    }

    void display() const override {
        Employee::display();
        cout << "Employee Type: Full-Time" << endl;
        cout << "Basic Salary: " << basicSalary << endl;
        cout << "Allowance: " << allowance << endl;
        cout << "Total Salary: "
             << calculateSalary() << endl;
    }
};

class PartTimeEmployee : public Employee {
private:
    int hoursWorked;
    double hourlyRate;
```

```
public:
    PartTimeEmployee(int id, string n,
                     int hours, double rate)
        : Employee(id, n) {
        hoursWorked = hours;
        hourlyRate = rate;
    }

    double calculateSalary() const override {
        return hoursWorked * hourlyRate;
    }

    void display() const override {
        Employee::display();
        cout << "Employee Type: Part-Time" << endl;
        cout << "Hours Worked: " << hoursWorked << endl;
        cout << "Hourly Rate: " << hourlyRate << endl;
        cout << "Total Salary: "
             << calculateSalary() << endl;
    }
};

int main() {
    cout << fixed << setprecision(2);

    FullTimeEmployee e1(101, "Arun", 30000, 5000);
    PartTimeEmployee e2(102, "Bala", 80, 200);

    Employee* employees[2] = {&e1, &e2};

    for (int i = 0; i < 2; i++) {
        employees[i]->display();
        cout << "--------------------" << endl;
    }

    return 0;
}
```

5. Expected output

```
Employee ID: 101
Name: Arun
Employee Type: Full-Time
Basic Salary: 30000.00
Allowance: 5000.00
Total Salary: 35000.00
--------------------
Employee ID: 102
Name: Bala
Employee Type: Part-Time
```

Hours Worked: 80
Hourly Rate: 200.00
Total Salary: 16000.00
---

6. Complexity analysis

Operation	Complexity
Calculate salary	\(O(1)\)
Display one employee	\(O(1)\)
Display \(n\) employees	\(O(n)\)
Store \(n\) employee references	\(O(n)\)

7. Result

The payroll hierarchy was designed using inheritance and runtime polymorphism. Full-time and part-time employees calculate their salaries using different rules while sharing a common interface.

Task 2: University People Hierarchy

1. Aim

To implement a university people hierarchy using inheritance and demonstrate multiple inheritance, constructor initialization, and the diamond problem.

2. Class hierarchy diagram

Person

Name and age

Student

Roll number

Employee

Employee ID

Student and Employee inherit virtually from Person

↓

TeachingAssistant

Student + Employee

3. Explanation

The diamond problem occurs when a class inherits from two classes that both inherit from the same base class. Without virtual inheritance, the derived class may contain two copies of the base-class data.

Using virtual public Person ensures that TeachingAssistant contains only one shared Person base subobject.

4. C++ program

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
   string name;
   int age;

public:
   Person(string n, int a) {
      name = n;
      age = a;
      cout << "Person constructor" << endl;
   }

   void displayPerson() const {
      cout << "Name: " << name << endl;
      cout << "Age: " << age << endl;
   }

   virtual ~Person() {
      cout << "Person destructor" << endl;
   }
};

class Student : virtual public Person {
protected:
   int rollNumber;

public:
   Student(string n, int a, int r)
      : Person(n, a) {
      rollNumber = r;
      cout << "Student constructor" << endl;
   }

   void displayStudent() const {
      cout << "Roll Number: " << rollNumber << endl;
   }
```

```
};

class Employee : virtual public Person {
protected:
   int employeeId;

public:
   Employee(string n, int a, int id)
      : Person(n, a) {
      employeeId = id;
      cout << "Employee constructor" << endl;
   }

   void displayEmployee() const {
      cout << "Employee ID: " << employeeId << endl;
   }
};

class TeachingAssistant : public Student, public Employee {
public:
   TeachingAssistant(string n, int a, int r, int id)
      : Person(n, a),
        Student(n, a, r),
        Employee(n, a, id) {
      cout << "TeachingAssistant constructor" << endl;
   }

   void display() const {
      displayPerson();
      displayStudent();
      displayEmployee();
   }

   ~TeachingAssistant() {
      cout << "TeachingAssistant destructor" << endl;
   }
};

int main() {
   TeachingAssistant t("Ravi", 22, 501, 701);

   cout << "\nUniversity Details" << endl;
   t.display();

   return 0;
}
```

5. Expected output

Person constructor

Student constructor
Employee constructor
TeachingAssistant constructor

University Details
Name: Ravi
Age: 22
Roll Number: 501
Employee ID: 701
TeachingAssistant destructor
Employee destructor
Person destructor

Correction for the program: The Employee class must also define its destructor to match the output above. Add this destructor at the end of the Employee class:

```
~Employee() {
    cout << "Employee destructor" << endl;
}
```

The actual constructor order is Person, Student, Employee, and TeachingAssistant. Destruction happens in reverse order. Because Person is a virtual base, it is destroyed last.

6. Result

Virtual inheritance prevents duplicate Person base-class instances. The program demonstrates multiple inheritance and the order in which constructors and destructors execute.

Task 3: Constructors, Destructors, and Object Lifetime

1. Aim

To understand parameterized constructors, member initialization, constructor execution order, and reverse destruction order in a class hierarchy.

2. Explanation

When an object of a derived class is created:

1. Virtual base classes are initialized first.
2. Direct base classes are initialized next.
3. Data members are initialized in their declaration order.
4. The derived-class constructor body executes last.

When the object is destroyed, the process occurs in reverse order.

3. Class diagram

Base

Base constructor and destructor

↓ Inherits from

Derived

Member object + derived constructor

4. C++ program

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base constructor" << endl;
    }

    ~Base() {
        cout << "Base destructor" << endl;
    }
};

class Member {
public:
    Member() {
        cout << "Member constructor" << endl;
    }

    ~Member() {
        cout << "Member destructor" << endl;
    }
};

class Derived : public Base {
private:
    Member m;

public:
    Derived() {
        cout << "Derived constructor" << endl;
    }

    ~Derived() {
```

```
        cout << "Derived destructor" << endl;
    }
};

int main() {
    cout << "Creating object" << endl;
    {
        Derived d;
        cout << "Object is active" << endl;
    }
    cout << "Object destroyed" << endl;

    return 0;
}
```

5. Expected output

```
Creating object
Base constructor
Member constructor
Derived constructor
Object is active
Derived destructor
Member destructor
Base destructor
Object destroyed
```

6. Explanation of output

- Base is constructed before Derived.
- The member object m is constructed before the body of Derived's constructor.
- When the scope ends, the Derived destructor executes first.
- Then Member is destroyed.
- Finally, Base is destroyed.

7. Result

The program demonstrates object construction and destruction order in C++. Base classes and member objects are constructed before the derived-class constructor body and destroyed in reverse order.

Task 4: Polymorphism, Base-Class Pointers, and Object Slicing

1. Aim

To demonstrate compile-time and runtime behavior in an inheritance hierarchy, including virtual functions, base-class pointers, and object slicing.

2. Explanation

Runtime polymorphism occurs when a virtual function is overridden in a derived class and called through a base-class pointer or reference.

Object slicing occurs when a derived-class object is copied into a base-class object by value. The derived-specific portion is lost in the resulting base object.

3. Class diagram

Shape

virtual display()

↓

Circle

display()

Rectangle

display()

4. C++ program

```
#include <iostream>
using namespace std;

class Shape {
public:
   virtual void display() const {
      cout << "This is a shape" << endl;
   }

   virtual ~Shape() {}
};

class Circle : public Shape {
public:
   void display() const override {
      cout << "This is a circle" << endl;
   }
};

class Rectangle : public Shape {
public:
   void display() const override {
      cout << "This is a rectangle" << endl;
   }
};
```

```
int main() {
    Circle c;
    Rectangle r;

    Shape* ptr;

    cout << "Runtime Polymorphism:" << endl;

    ptr = &c;
    ptr->display();

    ptr = &r;
    ptr->display();

    cout << "\nObject Slicing:" << endl;

    Shape baseObject = c;
    baseObject.display();

    return 0;
}
```

5. Expected output

```
Runtime Polymorphism:
This is a circle
This is a rectangle

Object Slicing:
This is a shape
```

6. Explanation

- ptr = &c calls the overridden Circle::display().
- ptr = &r calls the overridden Rectangle::display().
- Shape baseObject = c copies the base portion of c into a separate Shape object.
- The copied object calls Shape::display(), demonstrating slicing.

7. Result

The program demonstrates runtime polymorphism using virtual functions and base-class pointers. It also illustrates why passing derived objects by value as base objects can cause object slicing.

Task 5: Shopping Cart Class and the this Pointer

1. Aim

To implement a shopping cart using a C++ class and demonstrate encapsulation, member functions, and the this pointer.

2. Explanation

The this pointer refers to the current object on which a member function is called. It is useful when parameter names and member-variable names are identical.

For example:

this->quantity = quantity;

Here, this->quantity refers to the object's data member, while the second quantity refers to the function parameter.

3. Class diagram

Cart

Private data

- itemName: string
- price: double
- quantity: int

Public functions

- setItem()
- calculateTotal()
- display()

4. C++ program

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class Cart {
private:
   string itemName;
   double price;
   int quantity;

public:
   Cart() {
      itemName = "";
      price = 0;
      quantity = 0;
```

```
    }

    Cart& setItem(string itemName,
                  double price, int quantity) {
        this->itemName = itemName;
        this->price = price;
        this->quantity = quantity;
        return *this;
    }

    double calculateTotal() const {
        return price * quantity;
    }

    void display() const {
        cout << "Item: " << itemName << endl;
        cout << "Price: " << price << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Total Amount: "
             << calculateTotal() << endl;
    }
};

int main() {
    cout << fixed << setprecision(2);

    Cart cart;

    cart.setItem("Notebook", 50.00, 4);

    cout << "Shopping Cart Details" << endl;
    cart.display();

    return 0;
}
```

5. Expected output

```
Shopping Cart Details
Item: Notebook
Price: 50.00
Quantity: 4
Total Amount: 200.00
```

6. Complexity analysis

Operation	Complexity
Set item details	$O(1)$
Calculate total	$O(1)$

Display details \(O(1)\)

7. Result

The shopping cart was implemented using encapsulation and member functions. The this pointer distinguishes class data members from parameters, and returning *this allows method chaining.

For example, the following is also valid:

```
cart.setItem("Notebook", 50.00, 4)
   .display();
```

Task 6: Rental Fleet Management System

1. Aim

To design a reusable rental fleet hierarchy using inheritance and polymorphism to calculate rental charges for different types of vehicles.

2. Problem statement

A rental company manages cars and motorcycles. Each vehicle has a registration number and rental rate. The system must calculate rental charges according to the vehicle type and display vehicle details.

3. Class hierarchy diagram

Vehicle (Base Class)

Registration number, rental rate

virtual calculateRental()

↓ Inheritance

Car

Number of seats

Rental calculation

Motorcycle

Engine capacity

Rental calculation

4. C++ program

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class Vehicle {
protected:
   string registrationNumber;
   double dailyRate;

public:
   Vehicle(string reg, double rate) {
      registrationNumber = reg;
      dailyRate = rate;
   }

   virtual double calculateRental(int days) const = 0;

   virtual void display(int days) const {
      cout << "Registration Number: "
           << registrationNumber << endl;
      cout << "Daily Rate: " << dailyRate << endl;
      cout << "Rental Days: " << days << endl;
      cout << "Total Rental: "
           << calculateRental(days) << endl;
   }

   virtual ~Vehicle() {}
};

class Car : public Vehicle {
private:
   int seats;

public:
   Car(string reg, double rate, int s)
      : Vehicle(reg, rate) {
      seats = s;
   }

   double calculateRental(int days) const override {
      return dailyRate * days;
   }

   void display(int days) const override {
      cout << "\nCar Details" << endl;
      Vehicle::display(days);
      cout << "Seats: " << seats << endl;
   }
```

```
};

class Motorcycle : public Vehicle {
private:
   int engineCC;

public:
   Motorcycle(string reg, double rate, int cc)
      : Vehicle(reg, rate) {
      engineCC = cc;
   }

   double calculateRental(int days) const override {
      return dailyRate * days;
   }

   void display(int days) const override {
      cout << "\nMotorcycle Details" << endl;
      Vehicle::display(days);
      cout << "Engine Capacity: "
           << engineCC << " cc" << endl;
   }
};

int main() {
   cout << fixed << setprecision(2);

   Car car("TN01AB1234", 1500, 5);
   Motorcycle bike("TN02CD5678", 500, 150);

   Vehicle* fleet[2] = {&car, &bike};

   int days = 3;

   for (int i = 0; i < 2; i++) {
      fleet[i]->display(days);
   }

   return 0;
}
```

5. Expected output

```
Car Details
Registration Number: TN01AB1234
Daily Rate: 1500.00
Rental Days: 3
Total Rental: 4500.00
Seats: 5
```

Motorcycle Details
Registration Number: TN02CD5678
Daily Rate: 500.00
Rental Days: 3
Total Rental: 1500.00
Engine Capacity: 150 cc

6. Explanation

- Vehicle is an abstract base class because calculateRental() is a pure virtual function.
- Car and Motorcycle provide implementations of the rental calculation.
- The Vehicle* array stores pointers to different vehicle types.
- Calling display() through these pointers demonstrates runtime polymorphism.
- New vehicle types can be added later without changing the basic fleet interface.

7. Complexity analysis

Operation	Complexity
Calculate one vehicle's rental	\(O(1)\)
Display one vehicle	\(O(1)\)
Process \(n\) vehicles	\(O(n)\)
Store \(n\) vehicle pointers	\(O(n)\)

8. Result

The rental fleet management system was designed using abstraction, inheritance, and runtime polymorphism. The class hierarchy supports different vehicle types while keeping a common interface for rental calculations.

Final conclusion

This assignment demonstrates the major principles of object-oriented programming in C++:

1. Inheritance: Reusing common properties and functions from base classes.
2. Encapsulation: Protecting data through private and protected members.
3. Abstraction: Defining common interfaces using abstract classes.
4. Polymorphism: Calling derived-class implementations through base-class pointers.
5. Constructor and destructor order: Understanding object initialization and destruction.
6. Multiple inheritance: Combining features from more than one parent class.
7. Virtual inheritance: Preventing duplicate base-class instances in the diamond problem.
8. Object slicing: Understanding the loss of derived-class information when copying by value.
9. The this pointer: Referring to the current object within a member function.
10. Reusable class design: Adding new types without rewriting the whole system.